

§1 Algoritmos de Ordenação

§1.1 Insertion Sort

A implementação do Insertion Sort (`sortByScoreInsertion()`) foi feita da forma canônica, como nos slides da aula. O algoritmo percorre o array a partir do segundo elemento, inserindo cada jogador na posição correta em relação aos anteriores já ordenados. Apenas adicionamos a condição do *timestamp*, que ordena os jogadores com o mesmo score de acordo com a ordem de chegada. Essa condição é necessária, pois quando forem retirados grupos da fila de espera, poderia ocorrer instâncias onde a ordem de chegada não é preservada após o *sorting*.

§1.2 Merge Sort

A implementação do Merge Sort (`sortByScoreMerge()`) utilizou Divisão e Conquista, onde o array é dividido até conter apenas um elemento, e depois a função `merge(int left, int m, int right)` combina subarrays já ordenados utilizando dois subarrays auxiliares (`up` e `down`) alocados dinamicamente, respeitando a mesma lógica usada no Insertion do desempate por *timestamp*.

O uso da recursão é evidenciado pela implementação da função auxiliar `mergeAux(int left, int right)`, que chama a si mesma recursivamente para dividir o intervalo de índices pela metade (`m`) até que a condição base seja atingida (`left >= right`), representado um array de tamanho 1. O retorno das chamadas recursivas aciona a função `merge`, que agrupa as metades ordenadas.

§1.3 Formação de Grupos

A função `formGroup` explora diretamente o fato de que o array está ordenado para otimizar o seu tempo de execução. O algoritmo utiliza dois índices `head` e `tail`, onde `head + tail + 1 == groupSize` é verdade sempre, e checka a condição

```
players[tail].getScore() - players[head].getScore() <= delta
```

começando com `head = 0`, até encontrar um grupo válido. Isso drasticamente simplifica a implementação da função e diminui a sua complexidade computacional, como veremos em seguida.

§2 Análise de Custo Computacional

Aqui, N representa o tamanho da lista de jogadores.

- Inserção (`insert`): Insere o jogador na primeira posição livre do array de jogadores (`players[size]`), e atualiza o contador `size`. O custo computacional é $O(1)$, pois independe do quão grande o array de jogadores é (todas as operações na função são $O(1)$).
- Remoção (`removePlayer`): No pior dos casos, é necessário percorrer a lista inteira para encontrar ou não o id do jogador ($O(N)$). Depois, a atualização da lista é $O(N)$ também.
- Ordenação (`sortByScoreInsertion`): No pior caso (quando o array está em ordem decrescente de score), cada elemento precisa ser comparado e deslocado por todos os seus antecessores. Isso corresponde a $\binom{n}{2}$ operações, ou seja, o custo é $O(N^2)$.

- Ordenação (`sortByScoreMerge`): A complexidade é $O(N \log N)$ garantida, tendo em vista que a árvore de recursão possui altura $\log N$, e em cada nível da árvore a função `merge` percorre linearmente os N elementos para recombiná-los.
- Formação de Grupo (`formGroup`): Achar o grupo é um loop que percorre (no máximo) toda a fila linearmente ($O(N)$), e depois remover o intervalo (`removeInterval`) também é $O(N)$.
- Recuperação (`getWaitingPlayers`): Percorre toda a lista sempre, alocando um novo array de players e realizando a cópia exata dos N elementos na lista de espera ($O(N)$).

§3 Merge vs. Insertion

Os testes foram simulados usando a biblioteca `<chrono>` para medição de tempo de execução em microsegundos (μs). Para cada N , geramos o array de jogadores de forma tal que `players[i].getScore() = N - i`. Isso representa o pior caso para o Insertion Sort, e nos dará uma boa ideia de como o Merge é mais eficiente relativo ao Insertion, sem precisar utilizar bibliotecas para gerar scores aleatórios.

Table 1: Comparação de Tempo de Execução: Insertion Sort vs. Merge Sort

Tamanho (N)	Insertion Sort (μs)	Merge Sort (μs)	Razão (I/M)
10	6	18	0.33
50	124	86	1.44
100	219	88	2.48
500	5303	582	9.11
1000	22715	1280	17.74

Nota: Os valores representam o tempo em microsegundos no cenário de pior caso (array invertido). Para N grande, os dados ilustram o impacto do crescimento quadrático $O(N^2)$ contra o linear-logarítmico $O(N \log N)$.

Como apenas foram utilizados o cenário de pior caso para o Insertion Sort, é de se esperar que apenas nos casos onde N é pequeno o Insertion seria mais eficiente que o Merge, devido à sua maior complexidade de memória. Entretanto, quando N cresce, é visível o quanto a razão I/M cresce rapidamente.

Note também que os resultados são condizentes com as complexidades dos algoritmos. De fato, se o tempo de execução do Insertion para um tamanho N dura x , para ser condizente com a sua complexidade, o tempo de execução do Insertion para um array de tamanho kN deve ser $\sim k^2x$. Observando os dados da tabela, percebemos que isso acontece.

No caso do Merge, o tempo com um tamanho de kN deve ser $\sim kx \left(1 + \frac{\log k}{\log N}\right)$, o que pode ser verificado analisando a tabela que é o que acontece.